

Codex GPT-5.4 Reference vs Gemma4 31B Evaluation

Generated: 2026-06-19 12:58:45 UTC

Goal

This report evaluates the saved Gemma4 31B RAG answer run in docs/evaluations/answers/eval_v2_gemma4-31B.json against the Codex GPT-5.4 reference answers stored in docs/evaluations/eval_answers_v2.json. The format follows the older codex_vs_local_llm_evaluation_20260423 report style, but the reference side here is the curated Codex V2 answer file rather than a new source-reading pass.

Test Environment

Codex Reference Side

- Model: GPT-5.4 (Codex reference answers)
- Reference file: docs/evaluations/eval_answers_v2.json
- Question set: docs/evaluations/eval_questions_v2.json
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Local LLM Side

- Host: merlin-g-100.psi.ch
- Job / partition: 353392 on gwendolen
- Answer model: gemma4:31b
- Chunk explanation model: qwen2.5-coder:32b
- File / module / call-chain models: qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M
- Parser: tree_sitter
- Question count: 100
- Answer prompt mode: retrieval_answer_v2
- Mean answer latency: 41.00s
- Median answer latency: 37.58s

Retrieval Configuration Used by the Saved Run

- Candidate k: 20
- Supplementary k: 3
- Supplementary candidate k: 10
- Vector store chunk count: 4441
- Manifest embedding backend/model: ollama / nomic-embed-text

Important Caveat

The per-question verdicts are a structured comparison against the Codex reference answers, not an independent re-reading of every IPPL source file. A question is marked Correct when the local answer captures the central facts of the Codex reference; Partial when it contains relevant signal but misses important scope or implementation detail; and Incorrect when it abstains, points to the wrong subsystem, or omits the core reference answer.

Because the reference answers are intentionally concise, the grading emphasizes the main technical nouns, implementation locations, and algorithmic steps rather than exact wording. Ambiguous cases were treated conservatively as Partial rather than Correct.

Overall Result

Metric	Value
---	---
Questions	100
Correct	63
Partial	16
Incorrect	21
Average score (Correct=1, Partial=0.5, Incorrect=0)	0.710
Answers with leaked self-correction/planning text	0

Main Findings

- Strongest areas: Api Usage, File Purpose. These are mostly questions where the retrieved answer can be anchored to one well-described class, header, implementation file, or usage pattern.
- Weakest areas: Build And Install, Testing And Workflow. These questions require cross-file synthesis, precise implementation-location recovery, or careful numerical interpretation.
- Gemma4 31B is by far the slowest local run in this set, with mean latency 41.00s and median latency 37.58s.
- The larger Gemma model often gives comprehensive evidence lists, but the added verbosity does not always improve factual alignment with the concise Codex reference.
- Remaining misses are mostly retrieval/grounding failures: plausible evidence from adjacent IPPL components can still displace the exact implementation or numerical detail asked for.

Category Breakdown

Category	Questions	Correct	Partial	Incorrect	Avg score
---	---	---	---	---	---
File Purpose	13	9	3	1	0.808
Definition Location	15	11	0	4	0.733
Class Responsibility	13	9	1	3	0.731
Algorithm	12	8	2	2	0.750
Data Flow	9	4	3	2	0.611
Api Usage	12	9	3	0	0.875
Parallelism And Kokkos	9	4	3	2	0.611
Boundary And Halo	5	4	0	1	0.800
Numerical Meaning	3	2	0	1	0.667
Build And Install	5	1	1	3	0.300
Testing And Workflow	4	2	0	2	0.500

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: Correct
- Codex reference answer: Declares the top-level IPPL runtime handles (Comm, Env, Info, Warn, Error) and the lifecycle wrappers initialize, finalize, fence, and abort.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: Ippl.h is a central header file in the root module that provides

essential variables and functions for managing application initialization, termination, communication, and error handling. It defines global variables `ippl::Comm`, `ippl::Env`, `ipp...`

file_002 - What is the purpose of `Ippl.cpp`?

- Verdict: Partial
- Codex reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: `Ippl.cpp` provides essential utility functions within the `ippl` and `ippl::detail` namespaces used to manage initialization, finalization, and error handling for the IPPL framework. Its purpose includes implementing the following specific function...

file_003 - What does `FFT.hpp` implement?

- Verdict: Correct
- Codex reference answer: Implements `FFTBase` setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via `heFFT`.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.47
- Local answer excerpt: Answer: `FFT.hpp` implements a base class template `FFTBase<Field, FFT, Backend, T>` and various specializations of the `FFT<TemplateType, FieldType>` class for Fast Fourier Transform (FFT) operations within the `ippl` namespace. It supports real and comple...

file_004 - What is `BareField.h` responsible for?

- Verdict: Partial
- Codex reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.26
- Local answer excerpt: Answer: `BareField.h` defines the `ippl::BareField` class, which serves as a fundamental abstraction for representing fields in scientific computations within the IPPL framework. It is responsible for handling distributed data in scientific simulations by p...

file_005 - What does `Field.h` add on top of `BareField`?

- Verdict: Correct
- Codex reference answer: Adds mesh awareness and boundary-condition management on top of `BareField`, plus volume integral/average helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.56
- Local answer excerpt: Answer: `ippl::Field` adds mesh representation and boundary condition management to the basic field structure provided by `ippl::BareField`. Specifically, it utilizes mesh abstractions such as `UniformCartesian` and implements tag-dispatched boundary condit...

file_006 - What role does `HaloCells.h` play in IPPL?

- Verdict: Correct
- Codex reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: HaloCells.h defines the ippl::detail::HaloCells class and related structs used to manage halo (guard/ghost) cells for field data, facilitating inter-process communication (IPC) in parallel distributed-memory computations. Its primary role is to ...

file_007 - What is the purpose of FieldLayout.h?

- Verdict: Incorrect
- Codex reference answer: Defines how a global NDIndex domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: FieldLayout.h defines the ippl::FieldLayout class and associated helper functions used to manage field distribution, layout configurations, and domain information in scientific computations. It provides the bound_type struct for representing f...

file_008 - What does ParticleBase.h define?

- Verdict: Partial
- Codex reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.31
- Local answer excerpt: Answer: ParticleBase.h defines the ippl::ParticleBase class, which is a foundational abstraction for managing particle data in the IPPL framework. It also mentions the ippl::ParticleBaseBase class. Specifically, ParticleBase provides methods to hand...

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: Correct
- Codex reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: ParticleSpatialLayout.h defines the ippl::ParticleSpatialLayout class, which manages the spatial distribution of particles across processors based on their positions relative to a fixed grid. It is a specialized version of detail::ParticleLayout...

file_010 - What does ParticleAttrib.h implement?

- Verdict: Correct
- Codex reference answer: Implements the ParticleAttrib storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: ParticleAttrib.h implements the ippl::ParticleAttrib class template within the ippl namespace. This class is designed to manage and manipulate attributes associated with particles in a simulation—such as velocity and position—and provides func...

file_011 - What is UniformCartesian.h for?

- Verdict: Correct
- Codex reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: UniformCartesian.h defines the UniformCartesian class family within the ippl::Meshes namespace. This class inherits from Mesh<T, Dim> and is used to specialize uniform Cartesian meshes for numerical simulations in computational physics that ...

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: Correct
- Codex reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The file OrthogonalRecursiveBisection.h defines the ippl::OrthogonalRecursiveBisection class, which implements an orthogonal recursive bisection algorithm for decomposing computational domains in parallel computing. Its purpose is to divide comp...

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: Correct
- Codex reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: The purpose of CIC.h is to define function declarations within the ippl::detail namespace for Cloud-in-Cell (CIC) interpolation methods. These functions are used in numerical simulations employing particle-mesh techniques to distribute particle ...

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT initialization lives in src/FFT/FFT.hpp, especially FFTBase construction/setup and the transform-specific constructors that build the heFFTe plans.
- Comparison: The answer cites a different implementation location than the Codex reference.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: The retrieved context does not provide information on where FFT initialization is implemented in IPPL. The provided chunks instead cover general library initialization (ippl::initialize), field layout configuration (ippl::FieldLayout::initialize...

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: Incorrect
- Codex reference answer: IPPL sets up MPI in src/lppl.cpp::initialize() by constructing mpi::Environment and mpi::Communicator; the low-level MPI_Init check/call is in src/Communicate/Environment.cpp.

- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.24
- Local answer excerpt: Answer: The retrieved context does not provide information on where IPPL sets up the MPI environment. The provided chunks cover the initialization of FFT operations via `ippl::FFTBase::setup` using the HeFFTe library—which utilizes a communicator from Comm...

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: Correct
- Codex reference answer: Kokkos is initialized and finalized directly in `src/lppl.cpp` inside `ippl::initialize()` and `ippl::finalize()`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.75
- Local answer excerpt: Answer: Kokkos is initialized within the `ippl::initialize` function located in `lppl.cpp`, which calls `Kokkos::initialize(argc, argv)` using the remaining command-line arguments. The retrieved context does not explicitly state where Kokkos is finalized, a...

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: Correct
- Codex reference answer: The ORB load balancer is implemented in `src/Decomposition/OrthogonalRecursiveBisection.hpp` with declarations in the matching header.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.57
- Local answer excerpt: Answer: The orthogonal recursive bisection load balancer is implemented in the Decomposition module within the following files: - `data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.h`: Defines the class `ippl::OrthogonalRecursiveBisection`. - ...

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: Correct
- Codex reference answer: Field BCs are attached in `Field::setFieldBC()` and actually applied through `BConds::apply()` and the concrete face classes in `BcTypes.hpp`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: Boundary conditions are managed by the `ippl::Field` class via a member object `bc_m` of type `BConds_t`, which can be configured using the `setFieldBC` method in `src/Field/Field.h`. In Maxwell solvers, boundary conditions are applied post-update d...

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: Correct
- Codex reference answer: CIC scatter is implemented in `src/Interpolation/CIC.hpp`, primarily `scatterToPoint()` and `scatterToField()`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.82
- Local answer excerpt: Answer: The CIC (cloud-in-cell) scatter is implemented via the `scatterToField` and `scatterToPoint` functions within the `ippl::detail` namespace, located in `data/raw/ippl/src/Interpolation/CIC.hpp` and declared in `data/raw/ippl/src/Interpolation/CIC.h`....

loc_007 - Where is the CIC gather implemented?

- Verdict: Correct
- Codex reference answer: CIC gather is implemented in src/Interpolation/CIC.hpp, primarily gatherFromPoint() and gatherFromField().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.91
- Local answer excerpt: Answer: The CIC gather operations are implemented as the methods gatherFromField and gatherFromPoint within the ippl::detail namespace. These method definitions are located in the file src/Interpolation/CIC.hpp, while their corresponding function de...

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: Correct
- Codex reference answer: Halo exchange for BareField is surfaced in src/Field/BareField.hpp and actually performed in src/Field/HaloCells.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.69
- Local answer excerpt: Answer: Halo exchange for ippl::BareField is implemented via the ippl::detail::HaloCells class, which is defined as the halo_type within BareField. The communication of halo (boundary) data between different parts of a decomposed field across multip...

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: Correct
- Codex reference answer: The FFT open-boundary Poisson routines are defined in src/PoissonSolvers/FFTOpenPoissonSolver.h and src/PoissonSolvers/FFTOpenPoissonSolver.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.83
- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver routines are defined within the ippl::FFTOpenPoissonSolver class template. The class definition and declarations for its members (such as communicateVico, greensFunction, and initializeFields) are l...

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: Correct
- Codex reference answer: The CG/PCG algorithm implementation is in src/LinearSolvers/PCG.h; the Poisson wrapper that instantiates it is src/PoissonSolvers/PoissonCG.h.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The conjugate gradient solver algorithm is implemented in the ippl::CG class within the file data/raw/ippl/src/LinearSolvers/PCG.h. This class handles the iterative process of solving linear systems $Ax = b$, including the initialization and upd...

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: Correct
- Codex reference answer: The FEM Poisson solver is implemented in

src/PoissonSolvers/FEMPoissonSolver.h.

- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.83
- Local answer excerpt: Answer: The FEM Poisson solver is implemented in the FEMPoissonSolver class, located in the file data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h. This class inherits from Poisson<FieldLHS, FieldRHS> and solves the Poisson equation $-\nabla^2 u = f...$

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: Incorrect
- Codex reference answer: The FDTD Maxwell solver base class is defined in src/MaxwellSolvers/FDTSolverBase.h with implementation in FDTSolverBase.hpp.
- Comparison: The answer cites a different implementation location than the Codex reference.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The retrieved context does not provide information regarding where the FDTD Maxwell solver base class is defined. The provided chunks cover the FieldSolverBase abstract base class for field solvers (defined in src/Manager/FieldSolverBase.h), Car...

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: Correct
- Codex reference answer: The LagrangeSpace class is defined in src/FEM/LagrangeSpace.h with methods implemented in src/FEM/LagrangeSpace.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The ippl::LagrangeSpace class is defined in the file data/raw/ippl/src/FEM/LagrangeSpace.h. Additionally, its constructor and other implementations are located in data/raw/ippl/src/FEM/LagrangeSpace.hpp. Evidence: - data/raw/ippl/src/FEM/Lagra...

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: Correct
- Codex reference answer: Particle migration between ranks is implemented in ParticleSpatialLayout::update() in src/Particle/ParticleSpatialLayout.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The particle update routine responsible for moving particles across ranks is implemented in the update method within two different layout classes: 1. ippl::ParticleSpatialLayout<T, Dim, Mesh, Properties...>::update, located in data/raw/ippl/src...

loc_015 - Where are IPPL timers defined?

- Verdict: Incorrect
- Codex reference answer: IPPL timers are defined by the singleton timing layer in src/Utility/IpplTimings.h/.cpp, not just the low-level Timer helper.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: The retrieved context is insufficient to answer where IPPL timers are defined. The provided chunks cover how Timer::enableFences is configured via command-line arguments in ippl::initialize, and that communication logs second time stamps using s...

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: Correct
- Codex reference answer: BareField is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The `ippl::BareField` class is a fundamental abstraction used to represent fields in scientific computations within the IPPL framework. Its main responsibility is to manage memory and access patterns for multi-dimensional fields, specifically encaps...

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: Incorrect
- Codex reference answer: Field extends BareField with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The retrieved context is insufficient to answer this question as it does not contain information regarding a class named Field. The provided chunks focus exclusively on the `ippl::BareField` class, covering its role in managing memory and access patte...

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: Correct
- Codex reference answer: FieldLayout<Dim> distributes a global NDIndex<Dim> over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is Dim.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: The `ippl::FieldLayout` class, defined in FieldLayout.h and as a template in FieldLayout.hpp, manages the distribution of a multi-dimensional index space (represented by an NDIndex object) across MPI ranks. Its primary functions include perfor...

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: Correct
- Codex reference answer: ParticleBase is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The `ippl::ParticleBase` class is a foundational base class and manager used as a container for sets of particles in the IPPL framework. Its purpose is to manage particle data attributes, handle the creation and destruction of particles, and interfa...

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: Correct
- Codex reference answer: ParticleSpatialLayout decides which spatial region/rank owns each

- particle and performs the migration/update workflow after particles move.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
 - Reference-token coverage: 0.50
 - Local answer excerpt: Answer: The `ippl::ParticleSpatialLayout` class is a specialized version of `detail::ParticleLayout` that manages the spatial distribution and communication regions for particles based on their locations relative to a fixed grid. It ensures particles are ma...

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: Correct
- Codex reference answer: UniformCartesian represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: The UniformCartesian class, located in the `ippl::Meshes` namespace and inheriting from `Mesh<T, Dim>`, is responsible for representing and manipulating uniform-spacing Cartesian meshes used in scientific simulations. Its primary purpose is to enc...

cls_007 - What does the Mesh base class define?

- Verdict: Incorrect
- Codex reference answer: Mesh is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.13
- Local answer excerpt: Answer: The retrieved context is insufficient to answer this question. The provided chunks describe the CartesianCentering class, as well as the CCCEnums struct, CommonCartesianCenterings struct, and CenteringEnum enum used for managing centering lo...

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: Correct
- Codex reference answer: `ippl::mpi::Communicator` wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: The `ippl::mpi::Communicator` class, defined in `src/Communicate/Communicator.h` and implemented across `src/Communicate/Communicator.cpp` and `src/Communicate/Buffers.cpp`, is responsible for encapsulating MPI communication functionalities. Its pri...

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: Correct
- Codex reference answer: `ippl::mpi::Environment` manages MPI environment lifetime checks/teardown and provides an abort hook.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The `ippl::mpi::Environment` class, defined in `Environment.h` and implemented in `Environment.cpp`, manages the setup and teardown of the MPI (Message Passing

Interface) environment within the IPPL library. It provides functionality to initialize M...

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: Partial
- Codex reference answer: Archive serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.15
- Local answer excerpt: Answer: In the Communicate module, the `ippl::detail::Archive` class and its templated family `ippl::detail::Archive<Properties...>` are used to manage data buffers and handle serialization and deserialization operations for communication contexts. Defined ...

cls_011 - What is the role of the BConds class?

- Verdict: Correct
- Codex reference answer: BConds is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.56
- Local answer excerpt: Answer: The `ippl::BConds` class serves as a container for boundary condition (BC) objects associated with fields. Its role is to encapsulate the storage, management, and application of these boundary conditions to ensure numerical simulations respect physi...

cls_012 - What is the role of the BaseManager class?

- Verdict: Correct
- Codex reference answer: BaseManager provides the simulation loop skeleton: `pre_run`, `pre_step`, `advance`, `post_step`, and `run(nt)`.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.87
- Local answer excerpt: Answer: The `ippl::BaseManager` class is an abstract base class designed to manage simulations within the IPPL framework by providing a standardized structure for their lifecycle, including setup, execution, and cleanup phases. It implements the Template Me...

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: Incorrect
- Codex reference answer: Index represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.12
- Local answer excerpt: Answer: The retrieved context does not contain information about an Index class. The provided chunks describe a `constexpr` function named `index()` within a variant type structure that returns the index of the currently active type, and they detail the imp...

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic potential?

- Verdict: Partial
- Codex reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The FFTOpenPoissonSolver class template computes the electrostatic potential by solving the Poisson equation using Fast Fourier Transform (FFT) techniques and a Green's function approach based on the Vico-Greengard method, including an improved ve...

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: Correct
- Codex reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.45
- Local answer excerpt: Answer: IPPL implements FFT-based periodic Poisson solvers, such as the `ippl::FFTTruncatedGreenPeriodicPoissonSolver` and `ippl::FFTPeriodicPoissonSolver`, to solve equations of the form $\nabla^2 \phi = -\rho$ (and compute $E = -\nabla \phi$). The `ippl::...`

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic one?

- Verdict: Correct
- Codex reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: The `FFTTruncatedGreenPeriodicPoissonSolver` differs from the standard periodic solver (`FFTPeriodicPoissonSolver`) by utilizing truncated Green's functions to handle periodic boundary conditions effectively via Fast Fourier Transform (FFT) technique...

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: Incorrect
- Codex reference answer: In `PCG.h`, the CG iteration forms residual/direction fields, applies the operator, computes alpha and beta from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.27
- Local answer excerpt: Answer: The retrieved context does not provide the detailed internal implementation or mathematical iteration logic of the Conjugate Gradient algorithm within `PCG.h`. The provided chunks instead cover the setup and execution workflow: the iterative solver ...

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather directions)?

- Verdict: Correct

- Codex reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^Dim neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.43
- Local answer excerpt: Answer: Cloud-in-Cell (CIC) interpolation in IPPL is implemented through the scatter and gather methods of the `ippl::ParticleAttrib<T, Properties...>` class template, utilizing helper functions in the `ippl::detail` namespace defined in `CIC.h` and `CI...`

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: Correct
- Codex reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The standard FDTD solver advances the electromagnetic field in time through the `ippl::StandardFDTDSolver<EMField, SourceField, boundary_conditions>::step` method, which implements a leapfrog scheme based on a central difference method. This process...

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: Correct
- Codex reference answer: LagrangeSpace evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: The `ippl::LagrangeSpace` class determines the number of degrees of freedom per element using the function `getLagrangeNumElementDOFs`, which takes the spatial dimension (Dim) and polynomial order (Order) as parameters. This value is passed as a...

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: Correct
- Codex reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.58
- Local answer excerpt: Answer: Gauss-Jacobi quadrature is implemented via the `ippl::GaussJacobiQuadrature` class, which inherits from the abstract base class `ippl::Quadrature`. Its implementation of `computeNodesAndWeights()` is based on the LehrFEM++ library and uses Newton it...

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: Correct
- Codex reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.42

- Local answer excerpt: Answer: The `ippl::OrthogonalRecursiveBisection` class decides where to cut the domain using two primary criteria: the axis of the cut and the specific position along that axis. 1. Cut Axis: The method `findCutAxis` identifies the longest axis of the do...

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL?

- Verdict: Correct

- Codex reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.

- Comparison: The answer captures the main Codex reference concepts.

- Reference-token coverage: 0.53

- Local answer excerpt: Answer: In IPPL, inverse transform sampling is implemented via the `ippl::random::InverseTransformSampling::generate` method, which fills a Kokkos view with random samples. This process uses a target `Distribution` object—which encapsulates the probability ...

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: Incorrect

- Codex reference answer: The FFT layer builds `heFFTe` boxes/plans in `setup()`, copies IPPL field data into the required views/subviews, calls `heFFTe` forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.

- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.

- Reference-token coverage: 0.19

- Local answer excerpt: Answer: The FFT class interfaces with `heFFTe` through specialized transform types, including `CCTransform` for complex-to-complex and `RCTransform` for real-to-complex transforms. The interface process involves the following steps: 1. Domain Mapping: I...

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: Partial

- Codex reference answer: PCG applies a chosen preconditioner from `Preconditioner.h` to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.

- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.

- Reference-token coverage: 0.22

- Local answer excerpt: Answer: IPPL's preconditioned CG implementation utilizes the `ippl::CG` class template (defined in `PCG.h`), which provides a `setPreconditioner` method to configure the preconditioning strategy used during iterative solution processes. The preconditioners ...

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: Correct

- Codex reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (scatter) or FEM assembly (`assemble_rhs_from_particles`), accumulated into `rho`, and then normalized/background-shifted in the manager before solving.

- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.38
- Local answer excerpt: Answer: In a typical PIC deposition step, charge density flows from particles to the grid via the `assemble_rhs_from_particles` function. This process involves iterating over particles in parallel using a Kokkos loop where the following steps occur for each...

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: Partial
- Codex reference answer: After the field solve, the manager gathers the electric field back to particles with `CIC gather(...)` or `FEM interpolate_grad_to_diracs(...)`, storing particle-local E values for the push step.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.39
- Local answer excerpt: Answer: The retrieved context is insufficient to answer how the electric field flows from the grid back to particles. The provided chunks cover the assembly of the right-hand side (RHS) from particle attributes onto a field via `assemble_rhs_from_particles...`

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: Partial
- Codex reference answer: Halo exchange uses `FieldLayout` neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.26
- Local answer excerpt: Answer: Halo cells for a `BareField` are exchanged using the `ippl::detail::HaloCells` class, which is designed to manage guard/ghost cells and facilitate inter-process communication (IPC) by handling the transfer of boundary data between decomposed field p...

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: Correct
- Codex reference answer: `ParticleSpatialLayout::update()` applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.31
- Local answer excerpt: Answer: Particles are migrated between MPI ranks in the `ippl::ParticleSpatialLayout` class through a process triggered by the update routine, which is called after each time step where particle positions may have changed enough to affect the global layout...

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: Correct
- Codex reference answer: `FieldLayout` uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local `NDIndex`, plus neighbor and communication range tables.

- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.43
- Local answer excerpt: Answer: The retrieved context does not describe the specific logic or algorithm used by FieldLayout to decide which index ranges are assigned to which rank. The provided chunks do cover that the FieldLayout class manages field distribution, domain infor...

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: Correct
- Codex reference answer: ParameterList is merged into FFT and solver objects, then consumed by setup()/initialize()/solve() for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: In the FFT implementation, a ParameterList is passed to the constructor of classes like ippl::FFT<RCTransform, RealField> (defined in src/FFT/FFT.h). The constructor uses this list within a call to a setup() method, passing it along with Hef...

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during a simulation?

- Verdict: Partial
- Codex reference answer: BaseManager::run() executes pre_step(), advance(), and post_step() for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.48
- Local answer excerpt: Answer: The retrieved context is insufficient to answer how data flows through these hooks. The provided chunks cover the identity and lifecycle methods of the ippl::BaseManager class, the time-stepping mechanism in ippl::FDTDSolverBase, and the impleme...

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: Incorrect
- Codex reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (evaluateLoadVector, evaluateAx, FEMVector) and then interpolating the solved field quantities back where needed.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.20
- Local answer excerpt: Answer: The retrieved context is insufficient to answer how the FEM solver moves data between fields and FEMVector representations. Evidence: - data/raw/ippl/src/FEM/FEMVector.hpp : ippl::FEMVector<T>::accumulateHalo — explains moving data from local storag...

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: Incorrect
- Codex reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.

- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The retrieved context does not provide a detailed implementation or explanation of how particles are redistributed specifically *after* the rebalancing calculations (the recursive bisection loop). It does state that the `ippl::OrthogonalRecursiveBis...`

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: Partial
- Codex reference answer: A typical 3D field is `Field<double, 3, UniformCartesian<double, 3>, Cell>` constructed from a `UniformCartesian` mesh and a `FieldLayout<3>`.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: To create a 3D Field of doubles in IPPL, use the `ippl::Field` class template (which extends `BareField<T, Dim, ViewArgs...>`) specifying `double` for the type `T` and `3` for the dimension `Dim`. A field can be created using one of two methods: 1. ...

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: Correct
- Codex reference answer: Construct the mesh as `UniformCartesian<double, Dim> mesh(domain, hx, origin);` where `hx` is the spacing vector and `origin` is the mesh origin.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: To construct a `UniformCartesian` mesh with custom spacing and origin, you can use the constructor that takes the following three parameters: an `NDIndex<Dim>` object (`ndi`) representing the number of grid points along each dimension, a `vector_typ...`

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: Correct
- Codex reference answer: Create a `BConds<FieldT, Dim>`, assign per-face BC objects such as `PeriodicFace<FieldT>`, then call `field.setFieldBC(bc)`.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: To attach periodic boundary conditions to a field, you use the `setFieldBC()` method on the field object. In the `ippl::FDTD SolverBase` class, this is implemented in the `setPeriodicBoundaryConditions()` method, which performs the following steps: 1...

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: Correct
- Codex reference answer: Define a class derived from `ParticleBase<ParticleSpatialLayout<...>>`, add mass and velocity attributes as members, and register them with `addAttribute()` in the constructor.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.59
- Local answer excerpt: Answer: To add custom attributes such as mass and velocity to a particle

system based on `ippl::ParticleBase<PLayout, IP...>`, you use the `addAttribute` method. This method requires a reference to an attribute object of type `detail::ParticleAttribBase<Mem...`

api_005 - How do I call ippl::initialize and ippl::finalize in a main program?

- Verdict: Correct
- Codex reference answer: Call the free namespace functions `ippl::initialize(argc, argv)`; and `ippl::finalize()`; from main, as shown in the docs/examples.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: To initialize the IPPL library in a main program, call `ippl::initialize(int& argc, char* argv[], MPI_Comm comm)`, passing a reference to the command-line argument count (`argc`), the array of command-line arguments (`argv`), and an MPI communicator...

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: Partial
- Codex reference answer: Set up the relevant layouts/fields/parameters, build an `FFT<CCTransform, ...>` or `FFT<RCTransform, ...>` object, and call `transform(FORWARD, ...)` on the field(s).
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: To set up an FFT in IPPL, initialize an instance of the `ippl::FFT` class or its base class `ippl::FFTBase`. Setup is performed via constructors that take layouts and a `ParameterList`: * For specialized transforms like `ippl::FFT<RCTransform, RealF...`

api_007 - How do I construct an FFTPeriodicPoissonSolver and solve for the potential?

- Verdict: Correct
- Codex reference answer: Construct `FFTPeriodicPoissonSolver(lhs, rhs)` (or default construct then set fields/params), ensure the RHS is attached, and call `solve()` to write the potential/gradient outputs.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.52
- Local answer excerpt: Answer: To construct an `ippl::FFTPeriodicPoissonSolver<FieldLHS, FieldRHS>`, you can use its default constructor, which calls the base class constructor and invokes `setDefaultParameters()`; this process includes a `static_assert` to ensure that the value ...

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: Partial
- Codex reference answer: Use `PoissonCG<FieldT>` (or the Alpine `FieldSolver` wrapper around it), set its fields/parameters, and call `solve()` so the internal CG/PCG object iterates on the field.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: To invoke the conjugate gradient Poisson solver, use the `ippl::PoissonCG` class. You can instantiate it using either the default constructor or a constructor that takes references to the left-hand side (`lhs_type& lhs`) and right-hand side (`rhs_ty...`

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: Correct
- Codex reference answer: For Gaussian draws, IPPL exposes both randn convenience generation and the more general InverseTransformSampling + NormalDistribution path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.39
- Local answer excerpt: Answer: To sample particles from a normal distribution, use the `ippl::random::NormalDistribution` class in combination with the `ippl::random::InverseTransformSampling` class. First, initialize `NormalDistribution` by passing an array of parameters (`const...`

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: Correct
- Codex reference answer: Register each particle attribute with `addAttribute(...)`; migration then serializes and transfers all registered attributes automatically.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: To register particle attributes with `ippl::ParticleBase` for communication during migration, a user must define a class derived from `ParticleBase` and specify the desired attributes as instances of the `ParticleAttribute<T>` class. These attribute...

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: Correct
- Codex reference answer: Access rank and size with `ippl::Comm->rank()` and `ippl::Comm->size()` (or the same methods on a `Communicator` object).
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: You can access the MPI rank and size through the `rank()` and `size()` methods of the `ippl::mpi::Communicator` class. The `size()` method retrieves the total number of processes within the current MPI communicator, while the `rank()` method allows ...

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: Correct
- Codex reference answer: Get a timer handle with `IpplTimings::getTimer("name")`, surround the code with `startTimer()` / `stopTimer()`, and print with `IpplTimings::print()`.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.86
- Local answer excerpt: Answer: To measure a region of code using the singleton class `IpplTimings` (defined in `data/raw/ippl/src/Utility/IpplTimings.h`), follow these steps: 1. Create or retrieve a timer: Call `IpplTimings::getTimer("timer name")` to obtain an `IpplTimings::...`

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: Partial
- Codex reference answer: IPPL gets performance portability by combining Kokkos execution/memory

abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.

- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: IPPL achieves performance portability by integrating the Kokkos library for parallel execution capabilities. It utilizes Kokkos::View, which are described as performance-portable abstractions over arrays that can be used on different execution spa...

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: Correct
- Codex reference answer: IPPL uses Kokkos::View because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.27
- Local answer excerpt: Answer: The retrieved context does not explain why IPPL uses Kokkos::View for field storage. The provided chunks do cover that ippl::BareField utilizes a view_type (defined via detail::ViewType) as the actual storage for field data to handle memory ...

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: Correct
- Codex reference answer: Loops are parallelized through ippl::getRangePolicy / createRangePolicy and the ippl::parallel_for wrapper over Kokkos RangePolicy/MDRangePolicy.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: IPPL parallelizes loops over field indices by utilizing the ippl::RangePolicy struct to encapsulate Kokkos' multi-dimensional range management. The core of this system is the policy_type, which is defined as Kokkos::MDRangePolicy<PolicyArgs,...,...

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: Incorrect
- Codex reference answer: Precision is chosen by the field value type (Field<float,...> vs Field<double,...>), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: The retrieved context is insufficient to answer how IPPL supports mixed precision for fields. The chunks cover the following topics: - Initialization, finalization, and error handling in Ippl.cpp and Ippl.h. - Particle boundary condition impleme...

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same build?

- Verdict: Incorrect
- Codex reference answer: Multiple execution spaces are enabled at build time through IPPL_PLATFORMS and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.28
- Local answer excerpt: Answer: The retrieved context does not provide information on how IPPL

handles multiple execution spaces such as OpenMP and CUDA in the same build; it only mentions that the `ippl::initialize` function initializes Kokkos with any remaining command-line argu...

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: Correct
- Codex reference answer: ParallelDispatch.h wraps Kokkos policies and functors, exposing RangePolicy, getRangePolicy, createRangePolicy, parallel_for, and parallel_reduce.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.94
- Local answer excerpt: Answer: ParallelDispatch.h provides abstractions and utilities for parallel execution within the IPPL framework, specifically facilitating the dispatching and management of parallel tasks by leveraging the Kokkos library. It exposes the following helpers ...

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: Partial
- Codex reference answer: Archive serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and memcpy, then deserializes them back into views on receive.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The provided context is insufficient to answer how the Archive class serializes or deserializes Kokkos views for MPI transfer. The retrieved chunks only cover the definition of the `ippl::mpi` (or `mpi`) namespace across several files in the `src/Co...`

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: Correct
- Codex reference answer: MPI tags are managed with fixed base tags in Tags.h and the cyclic tag allocator in TagMaker, which Communicator inherits.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: MPI tags in IPPL are managed via an enumeration named Tag within the `ippl::mpi::tag` namespace, defined in `data/raw/ippl/src/Communicate/Tags.h`. This enum assigns integer constants to different types of messages to distinguish between various c...

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: Partial
- Codex reference answer: Kokkos::fence() waits for outstanding kernels; IPPL exposes ippl::fence() as a runtime wrapper in Ippl.cpp and also fences internally after many kernel/data-movement steps.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.43
- Local answer excerpt: Answer: The retrieved context does not provide information on what Kokkos::fence does; however, it describes ippl::fence as a synchronization point or barrier that ensures specific operations are completed before subsequent steps begin. IPPL calls the ippl:...

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: Correct
- Codex reference answer: BcTypes.h provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The file BcTypes.h defines a family of boundary condition classes derived from the base class ippl::detail::BCondBase. The provided boundary condition types are: - ippl::ConstantFace: Applies constant value boundary conditions. - ippl::Extrap...

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL Fields?

- Verdict: Incorrect
- Codex reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit NeumannFace class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The retrieved context does not provide information regarding Neumann boundary conditions; therefore, it cannot be compared to periodic boundary conditions. The provided text describes periodic boundary conditions for fields via the PeriodicFace cl...

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: Correct
- Codex reference answer: Ghost cells are populated mainly by fillHalo() / PeriodicFace::apply(): source slabs are packed, exchanged, and unpacked into ghost regions; accumulateHalo_noghost() is a different operation.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: Ghost cells are populated during a halo exchange via the ippl::detail::HaloCells<T, Dim, ViewArgs...>::exchangeBoundaries method defined in src/Field/HaloCells.hpp. The process follows these technical steps: 1. Range Identification: The meth...

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: Correct
- Codex reference answer: Periodic ghost cells are filled from opposite-side physical cells in applyPeriodicSerialDim() and the multi-rank PeriodicFace::apply() path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.37
- Local answer excerpt: Answer: In ippl::FDTDSolverBase, physical-cell values are copied into ghost cells during the applyBCs method, which calls .getFieldBC().apply() on the electromagnetic fields A_n, A_nm1, and A_np1 when boundary_conditions == periodic. These bou...

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing

waves?

- Verdict: Correct
- Codex reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The MaxwellSolvers module implements absorbing boundary conditions (ABC) to damp outgoing waves by minimizing reflections using second-order Mur ABCs. This is primarily managed by the second_order_mur_boundary_conditions struct in AbsorbingBC.h...

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: Correct
- Codex reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\Delta\phi = \rho$.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.69
- Local answer excerpt: Answer: The retrieved context is insufficient to answer what the FFT-based Poisson solver computes specifically in terms of Coulomb's law, as that term is not mentioned. The provided chunks instead describe how `ippl::FFTOpenPoissonSolver<FieldLHS, FieldRHS...`

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: Correct
- Codex reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E, B) driven by a source/four-current field.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.57
- Local answer excerpt: Answer: The FDTD solver advances electromagnetic fields (EMField) as part of a simulation of Maxwell's equations. Specifically, these solvers evaluate and update the electric field (E) and magnetic field (B) components. Evidence: - data/raw/ippl/src/Maxwe...

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The retrieved context does not provide information regarding the numerical tradeoffs between the FFT and CG Poisson solvers. The provided chunks describe the `ippl::CG` class as an iterative solver for linear systems $Ax = b$ that is effective for lar...

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: Partial
- Codex reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: IPPL depends on the following third-party libraries: - MPI: Used for parallel processing; specifically, the ippl::initialize function sets up the MPI environment and communicators using MPI_Comm. - Kokkos: Initialized within the ippl::i...

build_002 - What C++ standard does IPPL require, and why?

- Verdict: Incorrect
- Codex reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.10
- Local answer excerpt: Answer: The retrieved context does not provide information regarding the required C++ standard or the reasons for such a requirement. The provided chunks cover the initialization of the IPPL environment via ippl::initialize, the management of particle con...

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: Incorrect
- Codex reference answer: GPU builds are enabled through CMake options like IPPL_PLATFORMS=CUDA or IPPL_PLATFORMS="HIP;OPENMP" plus matching Kokkos/architecture/backend flags (Kokkos_ARCH_*, CMAKE_CUDA_ARCHITECTURES, Kokkos_ENABLE_HIP, Heffte_ENABLE_ROCM, etc.).
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.20
- Local answer excerpt: Answer: The retrieved context does not provide information on how IPPL is configured with CMake to enable a GPU build. Evidence: - data/raw/ippl/src/lpl.cpp : ippl::initialize — handles runtime initialization of Kokkos and parsing of command-line arguments...

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: Correct
- Codex reference answer: INSTALLATION.md describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and IPPL_PLATFORMS combinations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: INSTALLATION.md describes the following options and behaviors for configuring Kokkos: - Kokkos_VERSION: This option has a default value of 4.5.00. - Version Selection: By default, IPPL attempts to use versions found on the system. If a sys...

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: Incorrect
- Codex reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMP, clone the repo, configure a build,

compile, and test on an interactive node.

- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.10
- Local answer excerpt: Answer: The retrieved context is insufficient to answer how to install IPPL on the EULER cluster. The provided chunks cover the ippl::initialize function and environment setup in Ippl.cpp, particle management via ParticleBase and PicManager, finite ...

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member variables?

- Verdict: Correct
- Codex reference answer: WORKFLOW.md requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with _m.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: According to WORKFLOW.md, the following naming conventions are required: - Variables: Use camel casing (e.g., myVariable). - Compile time constants: Use capital casing (e.g., MY_CONSTANT). - Member variables: Must be suffixed with _m (e.g., m...

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: Correct
- Codex reference answer: Use std::sqrt in host-only code, and Kokkos::sqrt in device-capable code such as KOKKOS_INLINE_FUNCTION kernels.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.75
- Local answer excerpt: Answer: According to WORKFLOW.md, std::sqrt (and other standard library symbols) should be used for any instance of a mathematical function in host-only code. Conversely, Kokkos::sqrt (and other Kokkos symbols) must be used for instances occurring in de...

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: Incorrect
- Codex reference answer: IPPL profiles MPI calls with mpiP by preloading libmpiP.so, optionally setting MPIP flags, then parsing the produced .mpiP output with the bundled scripts/mpiP.py.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The provided retrieved context does not contain information on how IPPL profiles MPI calls using mpiP. The chunks cover MPI communication abstractions such as ippl::mpi::Communicator, ippl::mpi::Status, and ippl::mpi::Request, the retrieval of...

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: Incorrect
- Codex reference answer: The README does include a Gwendolen SLURM template showing the gmerlin6 cluster, gwendolen partition, GPU count, and srun ./<your_executable> <args> --kokkos-map-device-id-by=mpi_rank.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The retrieved context does not provide a SLURM job template for

Gwendolen; in fact, the term "Gwendolen" does not appear in the provided text. Evidence: -
data/raw/ipp/README.md : SLURM Job scripts — mentions that example job scripts are available ...